

Smali By bithowl: Chapter 12 Constants

“Before You Follow the Data, Find Where It Came From”



Small values...
Big clues...

SMALI BY BITHOWL : CHAPTER 12 CONSTANTS

“Before You Follow the Data, Find Where It Came From”

In the previous chapter, we learned how data moves through registers.
Now we learn how data actually moves.

```
const-string v0, "admin"
const/4 v1, 0x1
const-class v2, Lcom/example/
AdminActivity;
...
```

Where did this come from?



THE STORY (HOOK)

Imagine you're investigating an Android app. You enter a huge evidence room. There are thousands of boxes.

URLs

API Endpoints

Flags

Numbers

Class Names

Config Values

Strings

IDs

Each box has to come from somewhere. The const-* instructions are like workers placing evidence into numbered lockers.

→ A constant is the first clue. Follow it.

WHAT ARE CONSTANTS?

Instructions that load constant values into registers.

- const/4 → small integer (0,1,2,...)
- const/16 → 16-bit integer (e.g. 8080)
- const → 32-bit integer (e.g. 0x12345678)
- const-wide → long / double (two registers)
- const-string → String reference
- const-class → Class reference

The constant gives the value.
The register carries it.
The next instruction tells you what it means.

THE BIG PICTURE

Constant → Register

Then it flows...

- move
- invoke-*
- iget/put
- sget/sput
- if-*
- return

A constant is the beginning of a data-flow story.

1. const/4 - SMALL INTEGER

Loads a small integer into a register.

```
const/4 v0, 0x1
const/4 v1, 0x0
```

Meaning:
v0 = 1
v1 = 0

Often used for:

- booleans (0/1)
- flags
- small counters

1 = true, 0 = false

2. const/16 - 16-BIT INTEGER

Loads a 16-bit integer.

```
const/16 v0, 0x1F90
# 0x1F90 = 8080
```

Meaning:
v0 = 8080

Commonly used for:

- ports (8080)
- timeouts
- IDs / counters

Numbers can be clues (ports, timeouts, etc.).

3. const - 32-BIT INTEGER

Loads a 32-bit integer.

```
const v0, 0x12345678
```

Meaning:
v0 = 0x12345678

Commonly used for:

- large numbers
- masks
- version codes

Don't guess the meaning, follow its usage.

4. const-wide - LONG / DOUBLE

Loads a wide value (two registers).

```
const-wide v0, 0x123456789
# occupies v0 + v1
```

Meaning:
v0 = v1 = long value

Used for:

- timestamps
- long values
- double values

Always consider the type, descriptor (J or D).

5. const-string - STRING REF

Loads a string reference.

```
const-string v0, "https://api.com"
```

Meaning:
v0 → "https://api.com"

Commonly used for:

- URLs
- API endpoints
- keys / tokens
- messages / config

Strings reveal behaviour and configuration.

6. const-class - CLASS REF

Loads a class reference.

```
const-class v0, Lcom/example/AdminActivity;
```

Meaning:
v0 → AdminActivity.class

Commonly used for:

- reflection
- dynamic loading
- component registration
- system services

A class reference is a clue, not a vulnerability.

REGISTER TYPES — QUICK REFERENCE

Java Type	Small Descriptor	Width	Notes
boolean	Z	1	true/false
byte	B	1	-128 to 127
char	C	1	Unicode
short	S	1	-32768 to 32767
int	I	1	whole numbers
float	F	1	floating point
long	J	2	64-bit value
double	D	2	64-bit value
void	V	—	no value
String	Ljava/lang/String;	1	object reference
Object	Lpkg/Class;	1	object reference
int[]	[I	1	array reference
String[]	[Ljava/lang/String;	1	array reference

Objects and arrays are references. They occupy one register (not the entire object).

EXAMPLES — SEE IT IN ACTION

- Integer**

```
const/4 v0, 0x1
# v0 = 1
```
- String**

```
const-string v0, "admin"
# v0 → "admin"
```
- Class**

```
const-class v0, Lcom/example/User;
# v0 → User.class
```
- Wide (long)**

```
const-wide v0, 0x12345678
# v0 + v1 → long
```
- Method Result**

```
invoke-virtual {v0}, Ljava/lang/String;::length()I
move-result v1 # v1 = int
```

DATA FLOW EXAMPLE

```
const-string v0, "https://api.com"
↓
move-object v1, v0
↓
invoke-static {v1}, Lcom/example/Network;::connect(Ljava/lang/String;)V
```

Follow the value, not just the instruction.

COMMON PATTERNS (BUG HUNTER)

- Hardcoded URLs
const-string "https://..."
- API Keys
const-string "Bearer ..."
- Ports / Timeouts
const/16 8080 / 0x1F40
- Flags
const/4 0x1 / 0x0
const-class Lcom/....
- Strings in Logs
const-string "debug"

A constant is a starting point, not a vulnerability.

PRACTICAL EXERCISE — CONVERT JAVA TO CONSTANTS

Java Code	Small Instruction	Register Value
int enabled = 1;	const/4 v0, 0x1	v0 = 1
int port = 8080;	const/16 v0, 0x1F90	v0 = 8080
int value = 0x12345678;	const v0, 0x12345678	v0 = 0x12345678
long id = 123456789L;	const-wide v0, 0x75BCD15	v0/v1 = long
String url = "https://api";	const-string v0, "https://api"	v0 = ref
Class<T> c = AdminActivity.class;	const-class v0, Lcom/.../AdminActivity;	v0 = class ref

REGISTER TRACKING EXAMPLE

```
const-string v0, "admin"
const/16 v1, 0x1F90
const-class v2, Lcom/example/AdminActivity;
```

Register	Value
v0	→ "admin"
v1	→ 8080
v2	→ AdminActivity.class

BUG HUNTER CHECKLIST

- What type is it? (int, long, String, object, etc.)
- Which register receives it? (v0, v1, v2, p0, p1, etc.)
- Where is it used? (move, invoke, input, etc.)
- Is it security-sensitive? (token, URL, password, etc.)
- Is it a configuration value? (port, timeout, flag, etc.)
- What happens next? (method call, field access, decision, etc.)

THE REAL SKILL: Follow the data.

Constant (Where it came from) → Register (Where it lives) → Instruction (How it moves) → Method / Field (Where it goes) → Behaviour (What it does)

FINAL THOUGHT

Constants aren't just numbers or strings. They're the first evidence in a data trail. Follow them.

By - bithowl

In the previous chapter, we learned how data moves through registers.

We followed:

`v0 → v1 → method()`

We learned:

`move`

`move-object`

`move-wide`

`move-result`

`move-result-object`

`move-result-wide`

`iget`

`iput`

`sget`

`sput`

But there is an important question we haven't answered yet.

Where does the data come from in the first place?

Consider this: `const-string v0, "admin"`

Where did "admin" come from?

Or:

`const/4 v1, 0x1`

What is 1 doing there?

Or:

`const-class v2, Lcom/example/AdminActivity;`

Why is the application loading a class reference?

These instructions are all about **creating or loading constant values into registers**.

And for a bug hunter, this is extremely useful.

Because sometimes the first clue to a vulnerability is sitting right inside a constant.

The Story Hook

Imagine you're investigating an Android application. You enter a huge evidence room. There are thousands of boxes.

Some contain:

URLs, API endpoints, Flags, numbers, class names, configuration values, strings, IDs

But every box has to come from somewhere.

The const-* instructions are like the workers placing evidence into numbered lockers.

For example:

```
const-string v0, "https://api.example.com"
```

means:

Evidence:

```
"https://api.example.com"
```



Register v0

Another:

```
const/4 v1, 0x1
```

means:

```
1
```



v1

And:

```
const-class v2, Lcom/example/AdminActivity;
```

means:

AdminActivity class reference



v2

So before you follow data through an application, learn to recognize **where that data enters the register system**.

What Are const Instructions?

Smali has several instructions for loading constants into registers.

The important ones in this chapter are:

```
const/4
```

```
const/16
```

const

const-wide

const-string

const-class

They may look similar, but they are used for different kinds of values and different constant sizes.

A simple mental model:

const/4



small integer

const/16



16-bit integer

const



32-bit integer

const-wide



long / double

const-string



String reference

const-class



Class reference

Let's break them down.

1. const/4

Let's start with the smallest one.

`const/4 v0, 0x1`

This loads a small integer constant into v0.

Conceptually: $v0 = 1$

Another: `const/4 v0, 0x0`

means: $v0 = 0$

You will see these constantly in Android applications.

Why /4?

The /4 refers to the compact encoding of the literal value and register operands.

The important beginner takeaway is:

`const/4` is used for small integer constants and is very compact.

It is commonly seen with values such as:

0

1

2

-1

and other values that fit its limited range.

Example — Boolean Values

Remember:

boolean

is represented by:

Z

A boolean is commonly represented as:

0 → false

1 → true

So you might see: `const/4 v0, 0x1`

followed by: `return v0`

Conceptually:

`return true;`

Likewise:

`const/4 v0, 0x0`

return v0

can represent:

return false;

Bug Hunter Perspective

This becomes interesting when a constant influences security logic.

For example:

const/4 v0, 0x1

return v0

You might encounter this inside a method that appears to answer something like:

isDebugEnabled()

isAdmin()

isAuthenticated()

isAllowed()

isRooted()

Don't conclude that the application is vulnerable just because you see 0x1.

Instead: **Follow where that value goes.**

For example:

1

↓

v0

↓

return

↓

caller

↓

security decision

Context matters.

2. const/16

Now suppose the value is larger.

Example:

const/16 v0, 0x100

Conceptually:

v0 = 256

const/16 is used for a 16-bit literal.

You may see values such as: const/16 v0, 0x3E8

which is: 1000

Or: const/16 v1, 0x1F90

which is: 8080

That second example should immediately look interesting to someone investigating networking.

8080

could be a port number.

Again: A constant is a clue, not automatically a vulnerability.

Numbers Can Reveal Application Behaviour

Suppose you find:

const/16 v0, 0x1F90

You convert:

0x1F90

↓

8080

Now you might ask:

Is this a network port?

Is it a timeout?

Is it an application-specific ID?

Is it simply a numeric constant?

Then look at how v0 is used.

For example:

const/16 v0, 0x1F90

invoke-direct {v1, v0}, Ljava/net/InetSocketAddress;-><init>(Ljava/lang/String;I)V

Now the context tells you much more.

The number is being passed as an integer parameter to a network address constructor.

3. const

Now we reach the normal 32-bit constant instruction.

Example: `const v0, 0x12345678`

Conceptually: `v0 = 0x12345678`

Unlike `const/4` and `const/16`, this form can represent a full 32-bit literal.

You might encounter: `const v0, 0x10000`

or: `const v1, 0x7FFFFFFF`

The exact value isn't the only thing that matters.

Ask: **Why is the application using this value here?**

IDs, Flags and Magic Numbers

Applications contain plenty of numeric constants.

For example: `const v0, 0x100`

could be:

a flag

an ID

a size

a timeout

a bit mask

an error code

The number itself doesn't tell you its meaning.

The surrounding instructions do.

For example:

`const v0, 0x100`

`and-int/2addr v1, v0`

Now you have a completely different clue. It may be acting as a **bit mask**.

This is why Smali analysis isn't simply: "Translate every instruction into Java."

It is: **Understand what the value is doing.**

4. const-wide

Now let's return to something we learned in Chapter 10.

Some values are **wide**.

For example:

long

double

These use two register slots.

For a long, you might see: const-wide v0, 0x123456789

Conceptually: v0/v1 → long value

Remember:

v0

v1

work together as one wide value.

Example — Timestamp

Suppose Java contains:

```
long timestamp = 17000000000L;
```

Smali may represent the constant using:

```
const-wide v0, 0x65920080
```

The exact encoding/value depends on the number, but the important concept is:

const-wide

↓

wide constant

↓

v0 + v1

Later you might see:

```
move-wide v2, v0
```

Now:

v0/v1

↓

v2/v3

The value is moving as a wide pair.

const-wide and Bug Hunting

Large numeric constants can be useful clues.

For example:

timestamps

expiration values

IDs

cryptographic parameters

file sizes

bit masks

configuration values

Suppose you find:

const-wide v0, 0x...

and then:

invoke-static {v0, v1}, Lcom/example/Token;->isExpired(J)Z

Now the constant is being used as a timestamp-like value.

That context is far more meaningful than the raw hexadecimal number.

5. const-string

Now we reach one of the most important instructions for Android reverse engineering.

const-string v0, "hello"

This loads a string reference into v0.

Conceptually: v0 → "hello"

For example: const-string v0, "https://api.example.com"

means: v0 → https://api.example.com

Strings Are Gold for Recon

When analyzing an APK, strings can reveal:

API endpoints

URLs

file paths

JSON keys

HTTP headers

feature names

debug messages

database names

permissions

error messages

configuration values

For example:

```
const-string v0, "Authorization"
```

Immediately worth investigating.

Or:

```
const-string v1, "Bearer "
```

Or:

```
const-string v2, "https://internal-api.example.com"
```

Now you've found potential data-flow starting points.

Example — API Endpoint

Suppose:

```
const-string v0, "https://api.example.com/v1/login"
```

```
invoke-static {v0}, Lcom/example/Network;->request(Ljava/lang/String;)V
```

Data flow:

```
"https://api.example.com/v1/login"
```

↓

```
v0
```

↓

```
request()
```

As a bug hunter, you can now investigate:

Where is this endpoint used?

What data is sent?

Is authentication required?

Is it an internal endpoint?

Does another component expose it?

The const-string instruction gave you the starting point.

Example — Hardcoded Secret

Consider: `const-string v0, "sk_test_example"`

Don't immediately declare: "Hardcoded secret!"

Instead ask:

What is this string?

Where is it used?

Is it a real credential?

Is it test data?

Does the server accept it?

Is it public by design?

Then follow:

String

↓

register

↓

method

↓

network/API

This is the same **follow-the-data** mindset from Chapter 11.

6. const-class

Now we have something slightly different.

`const-class v0, Lcom/example/AdminActivity;`

This loads a **Class reference** into v0.

Conceptually:

v0 → Class object representing AdminActivity

It does **not** create an instance of AdminActivity.

That's important.

Compare:

`new-instance v0, Lcom/example/AdminActivity;`

with:

`const-class v0, Lcom/example/AdminActivity;`

They are not the same.

new-instance vs const-class

new-instance

Creates an object:

```
new-instance v0, Lcom/example/User;
```

Conceptually:

$v0 \rightarrow \text{new User}()$

const-class

Gets a Class reference:

```
const-class v0, Lcom/example/User;
```

Conceptually:

$v0 \rightarrow \text{User.class}$

Think:

new-instance

↓

"Give me an actual object."

const-class

↓

"Give me information identifying this class."

Why Would an App Need a Class Reference?

You may see:

```
const-class v0, Lcom/example/MyService;
```

```
invoke-virtual {v1, v0}, Landroid/content/Context;-  
>getSystemService(Ljava/lang/Class;)Ljava/lang/Object;
```

Or:

```
const-class v0, Lcom/example/MyActivity;
```

followed by reflection-related operations.

This can reveal:

dynamic class loading

reflection

dependency injection

component registration

serialization

framework behaviour

Class references can therefore be useful navigation clues.

const-class and Android Components

Imagine finding: `const-class v0, Lcom/example/AdminActivity;`

You might ask:

Why is this Activity being referenced here?

Is it launched conditionally?

Is it protected?

Is it exported?

Does it contain administrative functionality?

The instruction itself isn't a vulnerability.

But it gives you a **class to investigate**.

Constants + Data Flow = Powerful Combination

Now let's combine everything.

Suppose you discover:

`const-string v0, "https://internal.example.com"`

`const-string v1, "Authorization"`

`invoke-static {}, Lcom/example/Session;->getToken()Ljava/lang/String;`

`move-result-object v2`

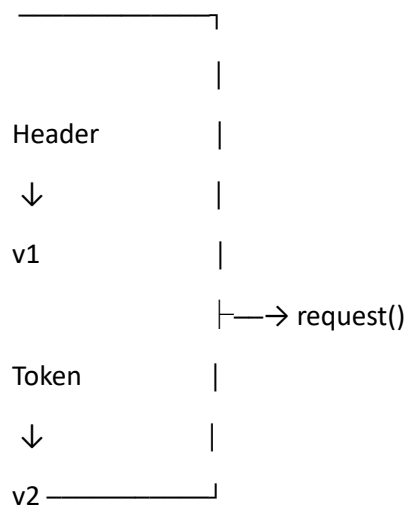
`invoke-static {v0, v1, v2}, Lcom/example/Network;-
>request(Ljava/lang/String;Ljava/lang/String;Ljava/lang/String;)V`

Look at the flow:

URL

↓

v0



Now you have three important pieces:

URL

Header

Token

That's much more useful than simply knowing:

“This method contains const-string.”

Bug Hunter Technique — Start From Interesting Constants

When opening an unfamiliar APK, interesting constants can become **anchors**.

Search for strings related to: http, https, api, token, auth, authorization, bearer, password, secret, admin, debug, internal, staging, localhost

Then inspect the surrounding Smali.

For numeric constants, look for: Ports, timeouts, Flags, permissions, IDs, bit masks, version checks, expiration values

For class constants:

AdminActivity

LoginActivity

WebViewActivity

DebugActivity

AuthService

NetworkService

The constant becomes your starting point.

Then:

Constant



Register



Move / field access



Method call



Security-sensitive behaviour

A Very Important Lesson: Hexadecimal Is Not the Meaning

Smali frequently displays numbers in hexadecimal.

For example: `const/4 v0, 0x1` is easy.

But you may encounter: `const v0, 0x7f080123`

Don't stare at the number and try to guess its meaning.

Ask: Where is `v0` used?

For example, it could represent a resource ID.

Similarly: `const/16 v0, 0x1f90`

becomes interesting only after you recognize:

`0x1F90 = 8080`

and then inspect its usage.

Register Tracking Exercise

Consider:

`const/4 v0, 0x1`

`const-string v1, "admin"`

`const/16 v2, 0x1F90`

`const-wide v3, 0x12345678`

`const-class v5, Lcom/example/AdminActivity;`

Build the register map.

Step 1

const/4 v0, 0x1

Therefore:

v0 → 1

Step 2

const-string v1, "admin"

Therefore:

v1 → "admin"

Step 3

const/16 v2, 0x1F90

Therefore:

v2 → 8080

Step 4

const-wide v3, 0x12345678

Therefore:

v3/v4 → wide value

Step 5

const-class v5, Lcom/example/AdminActivity;

Therefore:

v5 → AdminActivity.class

Final map:

v0 → 1

v1 → "admin"

v2 → 8080

v3/v4 → wide value

v5 → AdminActivity.class

Now you are ready to follow what happens to each value.

Practice — Convert Java Into Constants

Let's convert some simple Java statements.

Example 1

Java:

```
int enabled = 1;
```

Smali: const/4 v0, 0x1

Register: v0 → 1

Example 2

Java:

```
int port = 8080;
```

Smali: const/16 v0, 0x1F90

Register: v0 → 8080

Example 3

Java:

```
int value = 0x12345678;
```

Smali: const v0, 0x12345678

Register: v0 → 0x12345678

Example 4

Java:

```
long id = 123456789L;
```

Smali: const-wide v0, 0x75BCD15

Register: v0/v1 → long value

Example 5

Java:

```
String url = "https://api.example.com";
```

Smali: const-string v0, "https://api.example.com"

Register: v0 → URL reference

Example 6

Java:

```
Class<?> clazz = AdminActivity.class;
```

Smali: const-class v0, Lcom/example/AdminActivity;

Register: v0 → AdminActivity.class

Quick Reference

Instruction	What it loads	Typical use
const/4	Small integer	0, 1, flags
const/16	16-bit integer	ports, IDs, values
const	32-bit integer	larger integers, masks
const-wide	Wide constant	long, double
const-string	String reference	URLs, keys, messages
const-class	Class reference	reflection, class APIs

Bug Hunter Checklist

When you encounter a constant, don't just read it.

Ask six questions:

1. What type is it?

integer?

long?

double?

String?

Class?

2. Which register receives it?

v0?

v1?

v2/v3?

3. Where is that register used?

Look for:

move

invoke-*

iget/iput

sget/sput

if-*

return

4. Is the value security-sensitive?

Look for:

password

token

URL

API key

admin

debug

authentication

authorization

5. Is it a configuration value?

For example:

port

timeout

feature flag

environment

endpoint

version

6. What happens next?

This is the most important question.

constant

↓

register

↓

data flow

↓

method

↓

behaviour

Quick Recap

Smali gives us several ways to load constants.

const/4

For small integer values.

`const/16`

For 16-bit integer literals.

`const`

For 32-bit integer literals.

`const-wide`

For wide values such as long and double.

`const-string`

For String references.

`const-class`

For Class references.

And the most important thing to remember is: **A constant is the beginning of a data-flow story.**

Final Thought

When you open a Smali file, don't let the hexadecimal values scare you.

You might see:

`const/4 v0, 0x1`

`const/16 v1, 0x1F90`

`const-string v2, "https://api.example.com"`

`const-class v3, Lcom/example/AdminActivity;`

At first, it looks like random machine code.

But start translating:

1

↓

8080

↓

API URL

↓

AdminActivity

Now you're not just reading instructions.

You're beginning to understand what the application is trying to do.

And as a bug hunter, that's the real skill: **Don't just ask, "What constant is this?"**

Ask, “Why does the application need this value and where does it go?”

By - bithowl